

Sintade — Product & Architecture Document

2026-09-21 · @Someone

1. Overview

Sintade is a web-based platform that records screen, microphone, system audio and webcam in the browser, uploads while recording, and turns each recording into a shareable link that is playable seconds after stopping. Built from scratch: capture, upload, processing, playback and sharing are owned in-house; only rails (payments, email delivery, object storage hardware) are rented.

The name is a modified form of the Spanish *cinta de casete* (cassette tape): a recording you hand to someone.

Target users

Segment	Primary job	Example use
Individual professionals	Replace a meeting with a video	Bug report, code walkthrough, status update
Teams (5–200 seats)	Async communication in a shared library	Onboarding, design review, support replies
Educators and trainers	Lessons and tutorials	Course clips with captions
Support and sales	Personalised video replies	Customer walkthroughs with a CTA

First segment: developers and tech teams — bug reports, code walkthroughs, PR explanations and async standups. Other segments follow once this one retains. Plans are priced in TZS for the local market.

Goals

- Record-to-link in under 5 seconds after stop for recordings up to 30 minutes (streaming upload + fast-start remux).

- Zero lost recordings: every chunk persisted locally before upload; recoverable after tab crash, browser close or network loss.
- Playback in every modern browser, including Safari and mobile, via MP4/HLS output.
- Self-hostable core: Rust/Axum, Postgres, S3-compatible storage, FFmpeg — no mandatory third-party SaaS in the media path.
- Cost per stored recording-hour low enough for a usable free tier.

Non-goals (for now)

- Native desktop or mobile recording apps (browser-only capture; plus a companion browser extension built alongside the MVP).
- Live streaming or real-time video calls.
- A full timeline video editor (trim, cut and overlays only).
- Recording on mobile browsers (not supported by the platform APIs); mobile is view-only.

Glossary

Term	Meaning
Recording	The logical video a user creates; owns metadata, visibility and renditions
Take	One capture session; a recording has one take unless re-recorded or stitched
Chunk	A timesliced blob from <code>MediaRecorder</code> (1–5 s), the unit of local persistence and upload
Upload session	Server-side state tracking received chunks for one take
Rendition	A processed output: MP4, HLS ladder, thumbnail, sprite, captions
Share link	A slug plus access policy (visibility, password, expiry, download)
Workspace	A tenant: owns members, recordings, billing and policies

2. Tech stack and architecture principles

One Rust workspace with an API binary and a worker binary, one Postgres, one S3-compatible bucket, and an Angular SPA; everything else is added only when a measured need appears.

Layer	Choice	Why
Frontend	Angular (standalone components, signals), PWA; Manifest V3 browser extension (Chrome/Edge) sharing the capture package	Existing expertise; capture logic lives in framework-free TS services
Capture	<code>getDisplayMedia</code> , <code>getUserMedia</code> , Web Audio API, <code>MediaRecorder</code> , Canvas/OffscreenCanvas	Native browser APIs, no plugins
Local persistence	OPFS (fallback IndexedDB)	Durable chunk store that survives crashes
API	Rust, Axum, Tokio, tower-http, sqlx	Performance, type safety, low memory per connection
Database	Postgres 16+	Relational core, full-text search, <code>SKIP LOCKED</code> job queue, LISTEN/NOTIFY
Object storage	MinIO, self-hosted on a dedicated storage VPS (S3 API)	Full control, no per-GB storage fees, presigned URLs, lifecycle rules; the CDN absorbs egress
Media processing	FFmpeg + ffprobe invoked by Rust workers	Industry standard; remux, transcode, HLS, thumbnails
Transcription	whisper.cpp (self-hosted) via worker	No per-minute API cost; runs on CPU on the worker VPS
Playback	hls.js + native <code><video></code>	HLS on all browsers; native on Safari
CDN	Cloudflare or BunnyCDN	Media egress is the dominant cost
Email	SMTP via Amazon SES / Postmark / self-hosted	Transactional only

Layer	Choice	Why
Observability	tracing, OpenTelemetry, Prometheus, Grafana, Sentry	Logs, metrics, traces, errors
Deploy	Docker, GitHub Actions, VPS first	Matches current workflow; scale out later

Principles

1. **Modular monolith first.** One Cargo workspace, one crate per module, strict dependency rules (section 6). Split into services only when a module needs independent scaling — the media worker is the first and only planned split.
2. **Media never touches the API process.** Clients upload directly to object storage via presigned URLs; workers pull from storage. The API only handles metadata and signing.
3. **Chunks are the source of truth until processed.** A take is never "lost" as long as its chunks exist locally or in storage.
4. **Idempotent everything.** Chunk uploads, job execution and webhook deliveries can be retried safely; keys are (take_id, chunk_index) and job IDs.
5. **Postgres is the queue until it isn't.** SKIP LOCKED jobs table handles thousands of jobs/min; move to Redis/NATS only on evidence.
6. **Authorization at the edge of every resource.** Every media URL is signed and scoped; no public bucket.
7. **Ports and adapters.** Storage, email, payments, transcription and CDN sit behind traits so providers are swappable.
8. **Progressive capability.** Detect browser features at runtime and degrade clearly (e.g. no system audio on Safari) instead of failing silently.

```
#[async_trait]
pub trait ObjectStore: Send + Sync {
    async fn presign_put(&self, key: &str, ttl: Duration) -> Result<Url,
    StorageError>;
    async fn presign_get(&self, key: &str, ttl: Duration) -> Result<Url,
    StorageError>;
    async fn head(&self, key: &str) -> Result<Option<ObjectMeta>,
    StorageError>;
    async fn delete_prefix(&self, prefix: &str) -> Result<u64, StorageError>;
}
```

3. Feature areas (epics)

Twenty epics, each split into MVP (ship first), V1 (needed to compete) and V2 (differentiators and scale).

#	Epic	MVP	V1	V2
1	Capture	Screen/window/tab via <code>getDisplayMedia</code> ; mic with device picker; system/tab audio; mic + system mix via Web Audio; capability detection	Webcam bubble composited on canvas (shape, size, drag); camera-only mode; per-source volume and mute; live level meters; noise suppression / AEC / AGC; region crop	Background blur/replace; separate stored audio tracks; Document PiP control window; <code>preferCurrentTab</code> controls
2	Recording controls	Start, stop, pause, resume; 3-2-1 countdown; elapsed timer; click highlights and keystroke display via the extension; handle browser "Stop sharing" (<code>track.onended</code>)	Restart/discard take; keyboard shortcuts; plan max duration; quality presets (720p/1080p/4K, 30/60 fps, bitrate); cursor show/hide	On-screen drawing; scheduled recordings
3	Recording reliability	Timesliced chunks (2 s); OPFS/IndexedDB chunk store; crash recovery prompt	Streaming upload while recording; resumable retry with backoff; storage quota check; Wake Lock; <code>beforeunload</code> guard	Offline record-now, sync-later
4	Upload and ingest	Presigned PUT per chunk; upload session per take; SHA-256 per chunk; finalize endpoint	Idempotent retries; per-plan quota enforced at ingest; stale-session sweeper	S3 multipart for very long takes; multi-region ingest
5	Processing pipeline	Postgres job queue; concat chunks; fix WebM duration/cues; MP4 fast-start; thumbnail	HLS ladder 360p/720p/1080p; animated preview; scrub sprite; loudnorm; status via SSE	GPU workers; autoscaling; silence auto-trim
6	Playback	Player with play/pause/seek/volume/fullscreen; 0.5-2x speed	HLS + quality selector; captions (WebVTT); resume position; hover sprite; chapters; shortcuts	Embeddable iframe + oEmbed; branded player
7	Editing	—	Trim start/end; cut segments; non-destructive EDL rendered by FFmpeg; title/description	Stitch recordings; text overlays; blur/redact regions; zoom/pan; transcript-based cuts; re-record segment
8	Transcription and AI	—	Whisper transcription; VTT/SRT captions; searchable transcript with click-to-seek	Summary, auto-title, auto-chapters; filler-word removal; caption translation; Q&A over video
9	Sharing and access	Unguessable slug; private / link / public visibility	Workspace-only; password; expiry; invite by email; download toggle; revoke/regenerate; Open Graph + oEmbed	Signed expiring segment URLs; domain-restricted embeds; viewer watermark
10	Viewer engagement	—	Timestamped comments; emoji reactions; threads and @mentions; view counts	Viewer list with watch %; end-screen CTA; anonymous name capture
11	Library and organization	My recordings list; rename; delete	Folders; search title + transcript; filters/sort; trash with 30-day restore; bulk actions	Tags; duplicate; retention policies

#	Epic	MVP	V1	V2
12	Accounts and auth	Email/password (argon2id); email verification; password reset; session + refresh rotation	OAuth (Google, Microsoft, GitHub); profile; TOTP 2FA; active sessions list	SAML/OIDC SSO; SCIM
13	Workspaces and teams	Personal workspace auto-created	Multi-workspace membership; roles owner/admin/member/viewer; invites; shared library	Team spaces; admin policies; audit log
14	Billing and plans	Free-tier limits hard-coded (50 recordings, 10 min each, up to 1080p)	TZS plans + seats; direct mobile money APIs (M-Pesa, Airtel Money, Mixx by Yas, HaloPesa); usage metering; proration; invoices	Trials, coupons, annual; dunning
15	Notifications	Email: recording ready	In-app centre; comment/mention/first-view emails; per-user preferences	Digests; web push
16	Integrations and API	Browser extension (Chrome/Edge, MV3): record from any tab, click highlights, keystroke overlay; reuses the capture package	—	Public REST API + scoped keys; webhooks; Slack unfurl; Jira/GitHub/Notion/Drive; Zapier/Make
17	Analytics	—	Views, unique viewers, average watch time	Retention curve; workspace dashboard; CSV export
18	Security and compliance	HTTPS + HSTS; authz on every media request; rate limiting; size limits; CSRF; secure cookies	Encryption at rest; ffprobe validation; abuse reports; data export and deletion (Tanzania PDPA 2022)	NSFW moderation; data residency; SOC 2 readiness
19	Infrastructure and operations	Object storage; Postgres; separate worker process	CDN; tracing/metrics/Sentry; health checks; DLQ; raw-chunk lifecycle; backups (PITR)	Horizontal scaling; multi-region; per-tenant cost tracking
20	UX, accessibility, client	Permission onboarding; unsupported-browser messaging	Device check; dark/light; responsive viewer; WCAG 2.2 AA; PWA install	English + Swahili i18n; offline recording

Browser support matrix (capture)

Capability	Chrome/Edge (Win)	Chrome/Edge (macOS/Linux)	Firefox	Safari
Screen/window/tab	Yes	Yes	Yes (no tab)	Yes (no tab)
System audio	Full screen + tab	Tab only	No	No

Capability	Chrome/Edge (Win)	Chrome/Edge (macOS/Linux)	Firefox	Safari
MediaRecorder format	WebM VP8/VP9 + Opus; MP4 in recent versions	Same	WebM	MP4 H.264 + AAC
OPFS	Yes	Yes	Yes	Yes
Recording on mobile	No	No	No	No

4. Domain model and context map

The 20 epics collapse into 12 bounded contexts; Capture→Ingest→Media→Delivery is the core domain and where build effort concentrates.

Bounded contexts

Context	Type	Owns (aggregates)	Epics served
Capture	Core (client-only)	CaptureSession, SourceConfig, LocalChunk	1, 2, 3
Ingest	Core	UploadSession, ChunkReceipt	3, 4
Media	Core	Recording (media state), Take, Rendition, Job	5, 7
Delivery	Core	PlaybackGrant (signed URLs), Manifest	6
Catalog	Supporting	Recording (metadata), Folder, Tag, TrashEntry	11
Sharing	Core	ShareLink, AccessPolicy, Invite	9
Engagement	Supporting	Comment, Reaction, View	10, 17
Intelligence	Supporting	Transcript, Caption, Summary, Chapter	8
Identity	Generic	User, Credential, Session, OAuthLink, TotpSecret	12
Tenancy	Supporting	Workspace, Membership, Role, Policy	13

Context	Type	Owns (aggregates)	Epics served
Billing	Generic	Plan, Subscription, UsageRecord, Invoice	14
Messaging	Generic	Notification, Preference, Webhook, Delivery	15, 16

Platform concerns (epics 18, 19, 20) are cross-cutting libraries, not contexts.

Context map

flowchart LR

```

CAP[Capture<br/>client] -->|OHS: upload protocol| ING[Ingest]
ING -->|event: TakeFinalized| MED[Media]
MED -->|event: RenditionReady| DEL[Delivery]
MED -->|event: AudioReady| INT[Intelligence]
CAT[Catalog] -->|customer/supplier| MED
SHR[Sharing] -->|conformist| CAT
DEL -->|asks: canView?| SHR
ENG[Engagement] -->|conformist| SHR
TEN[Tenancy] -->|shared kernel: IDs| IDN[Identity]
BIL[Billing] -->|ACL over PSPs| TEN
ING -->|asks: quota| BIL
MSG[Messaging] -->|subscribes to events| MED

```

Arrows point from downstream to the upstream it depends on, or along the direction events flow; labels name the DDD relationship.

Relationship types

Upstream	Downstream	Relationship	Contract
Ingest	Capture	Open Host Service + Published Language	Upload protocol v1 (section 9)
Ingest	Media	Domain event	TakeFinalized { take_id, chunk_count, mime }

Upstream	Downstream	Relationship	Contract
Media	Delivery, Intelligence, Messaging, Catalog	Domain events	<code>RenditionReady</code> , <code>RecordingReady</code> , <code>AudioReady</code> , <code>ProcessingFailed</code>
Catalog	Sharing, Engagement	Conformist	Reads <code>recording_id</code> , <code>owner_id</code> , <code>workspace_id</code>
Sharing	Delivery, Engagement	Customer/Supplier	<code>AccessDecision</code> <code>can_view(viewer,</code> <code>recording)</code>
Identity	all	Shared kernel	<code>UserId</code> , <code>SessionClaims</code> types in <code>core</code> crate
Tenancy	all	Shared kernel	<code>WorkspaceId</code> , <code>Role</code> , <code>Permission</code>
Billing	Ingest, Media, Tenancy	Customer/Supplier	<code>Entitlements {</code> <code>max_duration,</code> <code>storage_bytes, seats,</code> <code>max_resolution }</code>
Payment providers	Billing	Anti-corruption layer	<code>PaymentProvider</code> trait; provider webhooks translated to internal events
Transcription engine	Intelligence	Anti-corruption layer	<code>Transcriber</code> trait (<code>whisper.cpp</code> , API)

Recording lifecycle (Media aggregate)

```
stateDiagram-v2
    [*] --> Recording: capture started
    Recording --> Uploading: stop pressed
    Recording --> Abandoned: discarded
    Uploading --> Processing: TakeFinalized
    Processing --> Ready: renditions done
    Processing --> Failed: job exhausted retries
    Failed --> Processing: manual retry
    Ready --> Processing: edit applied
    Ready --> Trashed: delete
    Trashed --> Ready: restore within 30 days
    Trashed --> [*]: purge
    Abandoned --> [*]
```

Ready is reached as soon as the fast-start MP4 exists; HLS and captions attach later without changing state.

Ubiquitous language rules

- A **Recording** is never called a "video" in code; "video" means a media stream.
- A **Take** belongs to exactly one Recording; re-recording creates a new Take and supersedes the old.
- **Visibility** (private, workspace, link, public) lives on ShareLink, never on Recording.
- **Entitlements** are computed by Billing; other contexts never read plan names.

5. Module breakdown

Each bounded context is one Rust crate that owns its tables and exposes a service trait plus events; two thin binaries (`api` , `worker`) wire them together.

Backend workspace layout

```
sintade/
├─ Cargo.toml           # workspace
├─ crates/
│   ├─ core/            # shared kernel: ids, errors, clock, Money,
Entitlements, events
│   └─ platform/        # config, PgPool, ObjectStore, Mailer,
telemetry, job queue, outbox
```

```
|  └ identity/
|  └ tenancy/
|  └ billing/
|  └ catalog/
|  └ ingest/
|  └ media/
|  └ delivery/
|  └ sharing/
|  └ engagement/
|  └ intelligence/
|  └ messaging/
└ bin/
|  └ api/                # Axum routers, extractors, middleware
|  └ worker/             # job runner: FFmpeg, whisper, emails, webhooks
└ migrations/           # sqlx, one folder prefix per module
  └ web/                 # Angular app
```

Each module crate follows the same internal shape: `domain/` (entities, invariants, no I/O), `app/` (service trait + impl, commands, queries), `infra/` (sqlx repositories, adapters), `events.rs` (published events).

Backend modules

Crate	Responsibility	Public interface	Owns tables
core	Shared kernel types and the event envelope	UserId, WorkspaceId, RecordingId, AppError, DomainEvent, Entitlements	—
platform	Infrastructure ports and adapters	ObjectStore, Mailer, JobQueue, Outbox, Clock, telemetry::init	jobs, outbox_events
identity	Sign-up, login, sessions, OAuth, 2FA, password reset	IdentityService: register, login, refresh, logout, verify_email, enable_totp; SessionClaims extractor	users, credentials, sessions, oauth_links, email_tokens, totp_secrets
tenancy	Workspaces, members, roles, policies	TenancyService: create_workspace, invite, accept, set_role; authorize(claims, Permission, WorkspaceId)	workspaces, memberships, workspace_invites, workspace_policies
billing	Plans, subscriptions, usage, entitlements	BillingService::entitlements(workspace), record_usage, provider webhook handler; PaymentProvider trait	plans, subscriptions, usage_records, invoices, payment_events
catalog	Recording metadata, folders, tags, trash, search	CatalogService: create_recording, rename, move_to_folder, trash, restore, search	recordings, folders, tags, recording_tags
ingest	Upload sessions, chunk presigning, finalize	IngestService: open_session, presign_chunk, ack_chunk, finalize; emits TakeFinalized	takes, upload_sessions, chunks

Crate	Responsibility	Public interface	Owns tables
media	Processing jobs and renditions, edits	MediaService: enqueue_processing, apply_edit, renditions(recording); job handlers ProcessTake, BuildHls, RenderEdit	renditions, edits, media_jobs
delivery	Playback grants, signed URLs, HLS manifest rewriting	DeliveryService::grant(viewer, recording) -> PlaybackGrant	— (stateless; caches in memory)
sharing	Links, visibility, passwords, expiry, invites	SharingService: create_link, update_policy, revoke, can_view(viewer, recording)	share_links, share_invites, link_unlocks
engagement	Comments, reactions, views, analytics	EngagementService: comment, react, record_view, stats(recording)	comments, reactions, views, view_segments
intelligence	Transcripts, captions, summaries, chapters	IntelligenceService: transcript(recording), captions(lang); Transcriber trait	transcripts, transcript_segments, captions, summaries
messaging	Notifications, preferences, webhooks	MessagingService::notify, WebhookDispatcher; subscribes to domain events	notifications, notification_prefs, webhook_endpoints, webhook_deliveries

Frontend modules (Angular)

Feature	Responsibility	Key pieces
core	HTTP client, auth interceptor, error handling, capability detection	ApiClient, AuthInterceptor, CapabilityService
capture	Framework-free recording engine	SourceManager, AudioMixer, Compositor (canvas), ChunkRecorder, ChunkStore (OPFS), Uploader
recorder	Recording UI	Source picker, device check, countdown, control bar, recovery dialog
library	Browse and organise	List/grid, folders, search, trash, bulk actions
player	Watch page	hls.js wrapper, captions, speed, chapters, transcript panel

Feature	Responsibility	Key pieces
share	Share dialog	Visibility, password, expiry, invites, embed code
engage	Comments and reactions	Timeline markers, threads, mentions
editor	Trim/cut UI	Waveform, range handles, EDL builder
account	Auth and settings	Login, signup, 2FA, sessions, profile
workspace	Team admin	Members, roles, policies, billing pages
extension (separate build)	Manifest V3 extension for Chrome/Edge: start recording from any tab, click highlights, keystroke overlay	Service worker, popup launcher, content script for click/keystroke overlay, offscreen document running the shared capture package, auth handoff to the web app session

```
// capture/chunk-recorder.ts – core contract of the capture engine
export interface ChunkRecorder {
  start(stream: MediaStream, opts: { mimeType: string; timesliceMs: number;
bitsPerSecond: number }): void;
  pause(): void;
  resume(): void;
  stop(): Promise<{ chunkCount: number; durationMs: number }>;
  readonly chunks$: Observable<{ index: number; blob: Blob }>;
  readonly state$: Observable<'idle' | 'recording' | 'paused' | 'stopping'>;
}
```

6. Dependency graph

Dependencies flow strictly downward through five layers; sideways calls between feature modules go through service traits or domain events, never through another module's tables.

```
flowchart TD
    API[bin/api] --> FEAT
    WRK[bin/worker] --> FEAT
    subgraph FEAT[Feature modules]
        ING[ingest]
        MED[media]
        DEL[delivery]
        SHR[sharing]
        ENG[engagement]
        INT[intelligence]
        MSG[messaging]
    end
    end
    FEAT --> CAT[catalog]
    FEAT --> BIL[billing]
    CAT --> TEN[tenancy]
    BIL --> TEN
    TEN --> IDN[identity]
    IDN --> PLT[platform]
    PLT --> CORE[core]
```

Read top to bottom: a crate may depend only on crates below it. `core` depends on nothing internal.

Layers

Layer	Crates	May depend on
L0 Kernel	<code>core</code>	std + small crates (<code>uuid</code> , <code>time</code> , <code>serde</code> , <code>thiserror</code>)
L1 Platform	<code>platform</code>	L0
L2 Foundation	<code>identity</code> , <code>tenancy</code>	L0-L1 (tenancy → identity allowed)
L3 Domain base	<code>catalog</code> , <code>billing</code>	L0-L2
L4 Features	<code>ingest</code> , <code>media</code> , <code>delivery</code> , <code>sharing</code> , <code>engagement</code> , <code>intelligence</code> , <code>messaging</code>	L0-L3, plus the explicit edges below
L5 Binaries	<code>api</code> , <code>worker</code>	Everything

Allowed edges inside L4

From	To	How	Why
<code>delivery</code>	<code>sharing</code>	Trait <code>AccessPolicy</code>	Must check access before signing URLs
<code>delivery</code>	<code>media</code>	Trait <code>RenditionQuery</code>	Needs rendition keys
<code>engagement</code>	<code>sharing</code>	Trait <code>AccessPolicy</code>	Only viewers who can view may comment
<code>ingest</code> → <code>media</code>	—	Event <code>TakeFinalized</code>	Decoupled via outbox
<code>media</code> → <code>intelligence</code> , <code>messaging</code> , <code>catalog</code>	—	Events <code>AudioReady</code> , <code>RecordingReady</code> , <code>ProcessingFailed</code>	Decoupled via outbox
<code>sharing</code> → <code>messaging</code>	—	Event <code>InviteCreated</code>	Email invites

From	To	How	Why
engagement → messaging	—	Events CommentCreated , FirstView	Notifications

Feature-level dependencies (epics)

Epic	Hard dependencies (must exist first)
1 Capture	20 (capability detection)
2 Recording controls	1
3 Reliability	1, 2
4 Upload and ingest	3, 12, 19
5 Processing	4, 19
6 Playback	5, 9
7 Editing	5, 6
8 Transcription	5
9 Sharing	11, 12
10 Engagement	6, 9, 12
11 Library	4, 12
12 Auth	19
13 Workspaces	12
14 Billing	13
15 Notifications	12, event outbox (19)
16 Integrations	1, 3, 4, 12 (extension, MVP); 9, 15 (API, webhooks, integrations)
17 Analytics	10
18 Security	Cross-cutting; applies to all from day one
19 Infrastructure	— (foundation)

Epic	Hard dependencies (must exist first)
20 UX/client	— (foundation)

Build order

1. `core`, `platform` (config, pool, ObjectStore, JobQueue, outbox, telemetry)
2. `identity`, then `tenancy` (personal workspace only)
3. `catalog`, stub `billing` returning free-tier `Entitlements`
4. Client `capture` engine + `ingest`
5. `media` (process take → MP4 + thumbnail)
6. `sharing` + `delivery` + player
7. Everything in V1, in roughly the epic order above

Enforcement

- Cargo prevents cycles between crates at compile time.
- `cargo-deny` / a CI script checks each crate's `[dependencies]` against the layer table.
- Each module's tables use a prefix-free schema but are only queried from that crate; a CI grep fails if `infra/` SQL names another module's tables.
- Angular: `@nx/enforce-module-boundaries` or ESLint `import/no-restricted-paths` with the same layering (`core` ← `capture` ← feature UIs).

7. Component diagram

Five runtime containers: the Angular SPA, the Axum API, the media worker, Postgres and object storage behind a CDN; media bytes flow browser → storage → worker → storage → CDN and never through the API.

Containers (C4 level 2)

flowchart LR

```

    U[Creator browser<br/>Angular SPA] -->|REST + SSE| API[API<br/>Axum]
    U -->|presigned PUT chunks| S3[(Object storage<br/>S3-compatible)]
    V[Viewer browser] -->|watch page, grants| API
    V -->|HLS/MP4 via signed URL| CDN[CDN]
    CDN --> S3
    API --> PG[(Postgres)]
    API -->|presign| S3
    W[Media worker<br/>Rust + FFmpeg] -->|jobs SKIP LOCKED| PG
    W -->|read chunks, write renditions| S3
    W -->|emails| SMTP[Email provider]
    API -->|webhooks in| PSP[Payment providers]
  
```

Container	Tech	Scales by	Notes
SPA	Angular, served as static files from CDN	CDN	Capture engine runs here
API	<code>bin/api</code> , Axum on Tokio	Horizontal, stateless	2+ replicas behind a load balancer
Worker	<code>bin/worker</code> , spawns FFmpeg/whisper	Horizontal, by queue depth	CPU-heavy; separate host from API
Postgres	Postgres 16	Vertical, then read replica	PITR backups
Object storage	MinIO on a dedicated VPS	Add disks, then multi-node erasure-coded MinIO	Private bucket; lifecycle rules
CDN	Cloudflare / Bunny	Provider-managed	Token-authenticated URLs

API components (C4 level 3)

flowchart TD

```

    R[Router + middleware<br/>trace, CORS, rate limit] -->
    X[Extractors<br/>SessionClaims, Workspace]
    X --> H[Handlers per module]
    H --> S[Module services]
    S --> REP[Repositories<br/>sqlx]
    S --> OB[Outbox writer]
    S --> OS[ObjectStore port]
    REP --> PG[(Postgres)]
    OB --> PG
    OS --> S3[(Storage)]
    SSE[SSE hub<br/>LISTEN/NOTIFY] --> PG
  
```

Component	Responsibility
Router + middleware	<code>tower-http</code> trace, compression, CORS, request ID, <code>tower-governor</code> rate limit, body size limits, timeouts
Extractors	Decode session cookie/JWT into <code>SessionClaims</code> ; resolve <code>WorkspaceContext</code> and role
Handlers	Thin: validate input (<code>validator</code>), call one service method, map to DTO
Services	Business rules and transactions; one per module
Repositories	<code>sqlx</code> queries with compile-time checking (<code>query!</code>)
Outbox writer	Writes domain events in the same transaction as state changes
SSE hub	Listens on Postgres <code>NOTIFY</code> and pushes processing status to connected clients

Worker components

Component	Responsibility
Job poller	<code>SELECT ... FOR UPDATE SKIP LOCKED</code> with per-type concurrency limits
Outbox relay	Reads <code>outbox_events</code> , dispatches to in-process subscribers, marks delivered
Handlers	<code>ProcessTake</code> , <code>BuildHls</code> , <code>GenerateSprite</code> , <code>Transcribe</code> , <code>RenderEdit</code> , <code>SendEmail</code> , <code>DeliverWebhook</code> , <code>PurgeTrash</code> , <code>SweepStaleUploads</code>
FFmpeg runner	Spawns <code>ffmpeg</code> with timeouts, parses progress from <code>-progress pipe:1</code> , streams stdout
Scratch manager	Per-job temp dir on local NVMe, cleaned on completion or panic

```
#[async_trait]
pub trait JobHandler: Send + Sync {
    const KIND: &'static str;
    type Payload: DeserializeOwned + Send;
    async fn run(&self, ctx: &JobCtx, payload: Self::Payload) -> Result<(),
    JobError>;
    fn max_attempts(&self) -> u32 { 5 }
    fn timeout(&self) -> Duration { Duration::from_secs(30 * 60) }
}
```

Client capture pipeline

```

flowchart LR
    D[getDisplayMedia] --> C[Compositor<br/>canvas: screen + cam]
    CAM[getUserMedia cam] --> C
    MIC[getUserMedia mic] --> MX[AudioMixer<br/>Web Audio]
    SYS[display audio] --> MX
    C --> MS[Combined MediaStream]
    MX --> MS
    MS --> MR[MediaRecorder<br/>2 s timeslice]
    MR --> ST[ChunkStore<br/>OPFS]
    ST --> UP[Uploader<br/>retry queue]

```

Without a webcam the screen track goes straight to `MediaRecorder`, skipping the canvas to save CPU.

8. Data model

Postgres holds all metadata with UUIDv7 keys and `workspace_id` on every tenant-owned row; object storage holds all bytes under a key scheme that makes per-recording deletion a single prefix delete.

Entity relationships (MVP + V1 core)

```

erDiagram
    USERS ||--o{ MEMBERSHIPS : has
    WORKSPACES ||--o{ MEMBERSHIPS : has
    WORKSPACES ||--o{ RECORDINGS : owns
    USERS ||--o{ RECORDINGS : creates
    RECORDINGS ||--o{ TAKES : has
    TAKES ||--|| UPLOAD_SESSIONS : has
    UPLOAD_SESSIONS ||--o{ CHUNKS : receives
    RECORDINGS ||--o{ RENDITIONS : has
    RECORDINGS ||--o{ SHARE_LINKS : shared_by
    RECORDINGS ||--o{ COMMENTS : has
    RECORDINGS ||--o{ VIEWS : has
    RECORDINGS ||--o{ TRANSCRIPTS : has
    FOLDERS ||--o{ RECORDINGS : contains

```

Core schema (MVP)

```
-- identity
CREATE TABLE users (
  id          uuid PRIMARY KEY,
  email       citext UNIQUE NOT NULL,
  display_name text NOT NULL,
  email_verified boolean NOT NULL DEFAULT false,
  created_at  timestampz NOT NULL DEFAULT now()
);
CREATE TABLE credentials (
  user_id      uuid PRIMARY KEY REFERENCES users(id) ON DELETE CASCADE,
  password_hash text NOT NULL          -- argon2id PHC string
);
CREATE TABLE sessions (
  id          uuid PRIMARY KEY,
  user_id      uuid NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  refresh_hash bytea NOT NULL,        -- SHA-256 of refresh token
  family_id    uuid NOT NULL,        -- rotation family for reuse
detection
  user_agent   text,
  ip           inet,
  expires_at   timestampz NOT NULL,
  revoked_at   timestampz,
  created_at   timestampz NOT NULL DEFAULT now()
);

-- tenancy
CREATE TYPE member_role AS ENUM ('owner','admin','member','viewer');
CREATE TABLE workspaces (
  id          uuid PRIMARY KEY,
  name        text NOT NULL,
  is_personal boolean NOT NULL DEFAULT false,
  created_at  timestampz NOT NULL DEFAULT now()
);
CREATE TABLE memberships (
  workspace_id uuid REFERENCES workspaces(id) ON DELETE CASCADE,
  user_id      uuid REFERENCES users(id) ON DELETE CASCADE,
  role         member_role NOT NULL,
  PRIMARY KEY (workspace_id, user_id)
);

-- catalog + media
```

```

CREATE TYPE recording_state AS ENUM

('recording','uploading','processing','ready','failed','abandoned','trashed')
;
CREATE TABLE recordings (
    id                uuid PRIMARY KEY,
    workspace_id      uuid NOT NULL REFERENCES workspaces(id),
    owner_id          uuid NOT NULL REFERENCES users(id),
    folder_id         uuid,
    title             text NOT NULL,
    description        text,
    state             recording_state NOT NULL DEFAULT 'recording',
    duration_ms        integer,
    width             integer,
    height            integer,
    size_bytes         bigint,
    current_take       uuid,
    trashed_at         timestampz,
    created_at         timestampz NOT NULL DEFAULT now(),
    updated_at         timestampz NOT NULL DEFAULT now(),
    search_tsv         tsvector GENERATED ALWAYS AS
                        (to_tsvector('simple', coalesce(title,'') || ' ' ||
coalesce(description,''))) STORED
);
CREATE INDEX recordings_ws_created ON recordings (workspace_id, created_at
DESC) WHERE trashed_at IS NULL;
CREATE INDEX recordings_search ON recordings USING gin (search_tsv);

-- ingest
CREATE TABLE takes (
    id                uuid PRIMARY KEY,
    recording_id       uuid NOT NULL REFERENCES recordings(id) ON DELETE CASCADE,
    mime_type          text NOT NULL,                -- e.g. video/webm;codecs=vp9,opus
    has_system_audio   boolean NOT NULL,
    has_mic            boolean NOT NULL,
    has_camera         boolean NOT NULL,
    finalized_at        timestampz,
    created_at         timestampz NOT NULL DEFAULT now()
);
CREATE TABLE chunks (
    take_id           uuid REFERENCES takes(id) ON DELETE CASCADE,
    idx              integer NOT NULL,
    size_bytes        integer NOT NULL,

```

```

    sha256      bytea NOT NULL,
    received_at timestampz NOT NULL DEFAULT now(),
    PRIMARY KEY (take_id, idx)
);

-- renditions
CREATE TYPE rendition_kind AS ENUM
('mp4', 'hls', 'thumbnail', 'preview', 'sprite', 'captions', 'audio');
CREATE TABLE renditions (
    id          uuid PRIMARY KEY,
    recording_id uuid NOT NULL REFERENCES recordings(id) ON DELETE CASCADE,
    take_id     uuid NOT NULL REFERENCES takes(id),
    kind        rendition_kind NOT NULL,
    variant     text NOT NULL DEFAULT 'default',    -- '720p', 'en', ...
    storage_key text NOT NULL,
    size_bytes  bigint,
    meta        jsonb NOT NULL DEFAULT '{}',
    created_at  timestampz NOT NULL DEFAULT now(),
    UNIQUE (recording_id, take_id, kind, variant)
);

-- sharing
CREATE TYPE visibility AS ENUM ('private', 'workspace', 'link', 'public');
CREATE TABLE share_links (
    id          uuid PRIMARY KEY,
    recording_id uuid NOT NULL REFERENCES recordings(id) ON DELETE CASCADE,
    slug        text UNIQUE NOT NULL,              -- 12+ chars base62, 70+
    bits
    visibility   visibility NOT NULL DEFAULT 'link',
    password_hash text,
    allow_download boolean NOT NULL DEFAULT false,
    expires_at   timestampz,
    revoked_at   timestampz,
    created_at   timestampz NOT NULL DEFAULT now()
);

-- platform
CREATE TABLE jobs (
    id          uuid PRIMARY KEY,
    kind        text NOT NULL,
    payload     jsonb NOT NULL,
    run_at      timestampz NOT NULL DEFAULT now(),
    attempts    integer NOT NULL DEFAULT 0,

```

```

    max_attempts integer NOT NULL DEFAULT 5,
    locked_by     text,
    locked_until  timestampz,
    last_error    text,
    done_at       timestampz,
    dead_at       timestampz
);
CREATE INDEX jobs_ready ON jobs (kind, run_at) WHERE done_at IS NULL AND
dead_at IS NULL;

CREATE TABLE outbox_events (
    id          bigserial PRIMARY KEY,
    event_type   text NOT NULL,
    aggregate_id uuid NOT NULL,
    payload      jsonb NOT NULL,
    created_at   timestampz NOT NULL DEFAULT now(),
    dispatched_at timestampz
);

```

V1 tables (summary)

Table	Key columns	Notes
oauth_links	user_id, provider, subject	Unique on (provider, subject)
totp_secrets	user_id, secret_enc, recovery_codes_hash[]	Secret encrypted with app key
workspace_invites	workspace_id, email, role, token_hash, expires_at	
folders	workspace_id, parent_id, name	Adjacency list; max depth 5
edits	recording_id, edl jsonb, version	EDL = list of kept [start_ms, end_ms] ranges
comments	recording_id, author_id, parent_id, at_ms, body, deleted_at	Index (recording_id, at_ms)

Table	Key columns	Notes
reactions	recording_id, user_id, emoji, at_ms	
views	recording_id, viewer_id?, anon_id?, started_at, watched_ms, max_position_ms	Partition by month when large
transcripts / transcript_segments	recording_id, lang; segments start_ms, end_ms, text, tsv	GIN on segment tsv for search
plans, subscriptions, usage_records	workspace_id, plan_id, period, metric, quantity	Entitlements derived, cached 60 s
notifications, notification_prefs	user_id, kind, payload, read_at	

Object storage layout

```

ws/{workspace_id}/rec/{recording_id}/
  takes/{take_id}/chunks/{idx:06}.webm      # raw, deleted 7 days after
processing
  takes/{take_id}/source.webm                # concatenated original, cold
tier
  mp4/default.mp4                            # fast-start, H.264/AAC
  hls/master.m3u8
  hls/{360p|720p|1080p}/index.m3u8, seg_{n}.m4s, init.mp4
  img/poster.jpg, preview.webp, sprite_{n}.jpg, sprite.vtt
  captions/{lang}.vtt
  edits/{version}/...

```

Deleting a recording = delete rows (cascade) + `delete_prefix(ws/{w}/rec/{r}/)` in a `PurgeRecording` job.

9. API surface

REST under `/api/v1`, JSON bodies, cookie session for the SPA, bearer API keys for V2 public API; errors follow RFC 9457 problem+json.

Endpoints

Method	Path	Module	Phase	Purpose
POST	/auth/register	identity	MVP	Create user + personal workspace
POST	/auth/login	identity	MVP	Issue session + refresh cookies
POST	/auth/refresh	identity	MVP	Rotate refresh token
POST	/auth/logout	identity	MVP	Revoke session
POST	/auth/verify-email, /auth/password/forgot, /auth/password/reset	identity	MVP	Token flows
GET	/me	identity	MVP	Current user + workspaces
POST	/recordings	catalog + ingest	MVP	Create recording + take + upload session
POST	/takes/{id}/chunks/{idx}/url	ingest	MVP	Presigned PUT for one chunk
POST	/takes/{id}/chunks/{idx}/ack	ingest	MVP	Confirm chunk: size + SHA-256
POST	/takes/{id}/finalize	ingest	MVP	Declare final chunk count and duration
GET	/takes/{id}/status	ingest	MVP	Which chunk indexes the server has (resume)

Method	Path	Module	Phase	Purpose
GET	/recordings	catalog	MVP	List, filter, search, cursor pagination
GET / PATCH / DELETE	/recordings/{id}	catalog	MVP	Read, rename, trash
POST	/recordings/{id}/restore	catalog	V1	Restore from trash
GET	/recordings/{id}/events	media	V1	SSE: processing status
POST	/recordings/{id}/edits	media	V1	Submit EDL, triggers RenderEdit
POST / PATCH / DELETE	/recordings/{id}/links[/link]	sharing	MVP	Manage share links
GET	/s/{slug}	sharing + delivery	MVP	Public watch-page data (title, poster, access requirements)
POST	/s/{slug}/unlock	sharing	V1	Password check, sets unlock cookie
GET	/s/{slug}/playback	delivery	MVP	Signed MP4/HLS URLs, 15-minute TTL
POST	/s/{slug}/views, /s/{slug}/views/{id}/heartbeat	engagement	V1	View start + progress every 10 s
GET / POST	/recordings/{id}/comments	engagement	V1	Timestamped comments

Method	Path	Module	Phase	Purpose
GET	/recordings/{id}/transcript	intelligence	V1	Segments with timings
GET	/oembed?url=	sharing	V1	oEmbed JSON
CRUD	/workspaces , /workspaces/{id}/members , /invites	tenancy	V1	Team management
GET	/billing/entitlements , POST /billing/checkout	billing	V1	Plan limits, start payment
POST	/webhooks/payments/{provider}	billing	V1	Provider callbacks (signature verified)

Upload protocol v1

```
sequenceDiagram
    participant B as Browser
    participant A as API
    participant S as Storage
    B->>A: POST /recordings (mime, sources)
    A-->>B: recording_id, take_id
    loop every 2 s chunk
        B->>B: write chunk to OPFS
        B->>A: POST chunks/{idx}/url
        A-->>B: presigned PUT (5 min)
        B->>S: PUT chunk
        B->>A: POST chunks/{idx}/ack (size, sha256)
    end
    B->>A: POST finalize (count, duration)
    A->>A: verify all idx present, enqueue ProcessTake
    A-->>B: 202 processing
    B->>B: delete OPFS chunks
```

Rules: chunk index is 0-based and contiguous; re-uploading an acked index with the same hash is a no-op, a different hash is `409`. Presigned URL requests are batched (`?count=10`) on reconnect to limit round trips. `finalize` returns `422` with the missing

indexes if any are absent.

Domain events (outbox)

Event	Producer	Consumers
TakeFinalized	ingest	media
RecordingReady	media	catalog, messaging, engagement
RenditionReady { kind, variant }	media	delivery cache, SSE
AudioReady	media	intelligence
ProcessingFailed { reason }	media	catalog, messaging
TranscriptReady	intelligence	catalog (search), messaging
LinkCreated, InviteCreated	sharing	messaging
CommentCreated, FirstView	engagement	messaging
SubscriptionChanged	billing	tenancy, ingest (entitlements cache)

```
#[derive(Serialize, Deserialize)]
pub struct EventEnvelope<T> {
    pub id: Uuid,
    pub event_type: &'static str,
    pub aggregate_id: Uuid,
    pub workspace_id: WorkspaceId,
    pub occurred_at: OffsetDateTime,
    pub version: u16,
    pub data: T,
}
```

10. Key flows

Four flows carry the product: record, process, watch, and recover; each is designed so the user sees a working link within seconds and no failure loses data.

Record

1. `CapabilityService` checks `getDisplayMedia`, `MediaRecorder.isTypeSupported`, OPFS, system-audio support; UI hides unsupported options.
2. User picks sources; device check shows mic level and camera preview.
3. `getDisplayMedia({ video: { frameRate: 30 }, audio: true })` → browser picker.
4. Mixer: mic + display audio → one `MediaStreamDestination` track. Compositor only if camera is on.
5. `POST /recordings` creates recording, take and upload session.
6. 3-2-1 countdown, then `MediaRecorder.start(2000)`.
7. Each `dataavailable` → write to OPFS → enqueue upload → ack.
8. Stop (button or `track.onended`) → flush last chunk → wait for queue drain (progress bar) → `finalize`.
9. Redirect to the watch page, which shows "processing" via SSE, then plays.

```
const mimeType = [
  'video/webm;codecs=vp9,opus',
  'video/webm;codecs=vp8,opus',
  'video/mp4;codecs=avc1,mp4a',
].find(t => MediaRecorder.isTypeSupported(t));

const rec = new MediaRecorder(stream, { mimeType, videoBitsPerSecond:
2_500_000, audioBitsPerSecond: 128_000 });
let idx = 0;
rec.ondataavailable = async e => {
  if (!e.data.size) return;
  const i = idx++;
  await chunkStore.put(takeId, i, e.data); // OPFS first, always
  uploader.enqueue(takeId, i);
};
stream.getVideoTracks()[0].onended = () => rec.state !== 'inactive' &&
rec.stop();
rec.start(2000);
```

Process (worker, `ProcessTake`)

1. Download chunks to scratch in index order; verify SHA-256 against `chunks`.
2. Concatenate bytes (timesliced WebM chunks are one continuous stream) → `source.webm`.

3. `ffprobe` to validate container, codecs, duration; reject anything unexpected.
4. Produce fast-start MP4; if source is already H.264/AAC (Safari), remux only.
5. Poster at 1s; mark recording `ready`; emit `RecordingReady`.
6. Enqueue `BuildHls`, `GenerateSprite`, `Transcribe` as separate lower-priority jobs.

```
# 4: transcode VP9/Opus -> H.264/AAC, fast-start, fixes missing duration/cues
ffmpeg -y -i source.webm -c:v libx264 -preset veryfast -crf 23 -pix_fmt
yuv420p \
  -c:a aac -b:a 128k -af loudnorm -movflags +faststart default.mp4

# 5: poster
ffmpeg -y -ss 1 -i default.mp4 -frames:v 1 -vf scale=1280:-2 poster.jpg

# BuildHls: 720p variant (repeat per rung)
ffmpeg -y -i default.mp4 -vf scale=-2:720 -c:v libx264 -preset veryfast -crf
23 \
  -c:a aac -b:a 128k -hls_time 4 -hls_playlist_type vod \
  -hls_segment_type fmp4 -hls_segment_filename '720p/seg_%04d.m4s'
720p/index.m3u8
```

Target: `ProcessTake` finishes in $\leq 0.5 \times$ recording length at 1080p on 4 vCPU (`veryfast`).
 For under 5 s link-to-play, the watch page can serve `source.webm` directly to
 Chrome/Firefox while the MP4 builds.

Watch

1. `GET /s/{slug}` $\rightarrow$ title, poster, requirement (`none`, `login`, `password`, `workspace`).
2. If needed: login or `POST /s/{slug}/unlock`.
3. `GET /s/{slug}/playback` $\rightarrow$ `SharingService::can_view` $\rightarrow$
`DeliveryService::grant` $\rightarrow$ signed URLs (15 min).
4. Player picks HLS (`hls.js` / native Safari) or MP4 fallback.
5. `POST /views` on first play; heartbeat every 10 s with position; unique-view dedupe per
 viewer per 30 min.

Recover

1. On app load, `ChunkStore.listOrphans()` finds takes with local chunks not yet
 finalized.
2. Dialog: "Unfinished recording from 14:02, 6 min 20 s — Upload / Discard".

3. Upload: `GET /takes/{id}/status` → upload only missing indexes → `finalize` with local count.
4. Server side: `SweepStaleUploads` marks sessions idle > 24 h as `abandoned` and deletes their chunks after 7 days.

11. Non-functional requirements

Targets are set for the MVP on a single VPS pair (API + worker) and must hold until roughly 1,000 active creators.

Performance

Metric	Target
Stop → playable link (30 min recording, 10 Mbps uplink)	≤ 5 s p95
Recording ready as MP4 (1080p30)	≤ 0.5× duration p95
API latency, metadata endpoints	≤ 100 ms p95, ≤ 300 ms p99
Playback start (time to first frame)	≤ 1.5 s p75 on 10 Mbps
Client CPU during 1080p30 recording, no camera	≤ 25% on a 4-core laptop
Chunk upload overhead vs raw size	≤ 3%
Library list, 10k recordings in workspace	≤ 150 ms p95

Reliability

Requirement	Target / mechanism
Recording data loss	0 for any recording whose chunks reached OPFS
API availability	99.9% monthly
Job success after retries	≥ 99.9%; failures land in DLQ with alert
Retries	Exponential backoff 2^n s, max 5, jitter ±20%
Backups	Postgres PITR 14 days; nightly base backup off-site; storage versioning 7 days

Requirement	Target / mechanism
RPO / RTO	5 min / 1 h
Graceful shutdown	API drains 30 s; worker finishes or re-queues current job

Security

Area	Requirement
Passwords	argon2id, m=19 MiB, t=2, p=1 (OWASP baseline)
Sessions	Access token 15 min; refresh 30 days, rotated on use, reuse revokes family; <code>HttpOnly</code> ; <code>Secure</code> ; <code>SameSite=Lax</code>
CSRF	SameSite cookies + double-submit token on state-changing requests
Rate limits	Login 5/min/IP+email; signup 3/hour/IP; presign 600/min/user; public watch 120/min/IP
Media access	Private bucket; presigned URLs $\leq$ 15 min; CDN token auth; slugs $\geq$ 70 bits entropy
Uploads	Max chunk 16 MB; max take per plan; <code>ffprobe</code> validation; reject unknown codecs
Headers	CSP (no inline scripts), HSTS 1 year, <code>X-Content-Type-Options</code> , <code>Referrer-Policy: strict-origin-when-cross-origin</code> , <code>Permissions-Policy: display-capture=(self)</code>
Secrets	Env vars from secret store; TOTP secrets encrypted with AES-256-GCM
Privacy	Data export and account deletion within 30 days; deletion purges storage prefix (Tanzania PDPA 2022; GDPR-aligned)
Tenant isolation	Every query scoped by <code>workspace_id</code> ; integration tests assert cross-tenant access returns 404

Cost

Driver	Control
Storage	Delete raw chunks 7 days after processing; <code>source.webm</code> to cold tier after 30 days; HLS rungs only for recordings viewed ≥ 1 time (lazy ladder)
Egress	Cloudflare or Bunny CDN in front of MinIO so the storage VPS serves each segment about once; serve 720p by default
Compute	<code>veryfast</code> preset; transcode on demand for rarely watched rungs; whisper <code>base</code> / <code>small</code> model on CPU
Free tier	50 recordings, 10 min each, up to 1080p, 90-day retention for inactive

Accessibility and compatibility

- WCAG 2.2 AA for all UI; captions for every processed recording in V1.
- Recording: latest 2 versions of Chrome, Edge, Firefox, Safari (desktop).
- Viewing: all evergreen browsers, iOS Safari 16+, Android Chrome.
- Keyboard operable recorder and player; visible focus; screen-reader labels on controls.

Observability

- Structured JSON logs with request ID and `workspace_id`; no PII in logs.
- Metrics: request rate/latency/errors per route, queue depth per job kind, job duration, FFmpeg failures, upload chunk failures, storage bytes per workspace.
- Client telemetry: capture errors, `MediaRecorder` mime chosen, chunk upload retries, recovery-dialog shown.
- Alerts: job DLQ > 0, queue age > 5 min, API 5xx > 1% over 5 min, disk > 80% on worker scratch.

12. Roadmap

Three phases, sized for one developer: MVP in about 14 weeks including the browser extension, V1 in a further 16, V2 open-ended; durations are estimates, not commitments.

MVP — weeks 1-14: record, upload, share, watch, extension

Week	Milestone	Deliverables
1-2	Foundation	Cargo workspace, <code>core</code> , <code>platform</code> (config, pool, ObjectStore on MinIO, JobQueue, outbox, tracing); Angular shell; CI; Docker Compose dev env
3-4	Identity	Register, login, refresh rotation, logout, email verification, password reset; personal workspace
5-7	Capture engine	Capability detection, sources, mixer, <code>ChunkRecorder</code> , OPFS <code>ChunkStore</code> , controls, countdown, recovery dialog; capture kept as a standalone package
7-8	Ingest	Recording/take/session creation, presign, ack, status, finalize; streaming uploader with retry
9-10	Processing	Worker binary, <code>ProcessTake</code> (concat, ffprobe, MP4 fast-start, poster), state machine, email on ready
10-11	Share + watch	Share links (private/link/public), watch page, playback grants, basic player, library list
12-13	Browser extension	MV3 extension for Chrome/Edge: launch from any tab, click highlights, keystroke overlay, session handoff from web app; store listing
14	Hardening	Rate limits, security headers, tenant-isolation tests, MinIO + Postgres backups, deploy to production VPS, 10 real developer users

Exit criteria:

- ☐ 30-minute 1080p recording on Chrome, Edge, Firefox, Safari uploads and plays everywhere
- ☐ Killing the tab at minute 10 and reopening recovers the recording with no gaps

- ☐ Dropping network for 60 s mid-recording loses nothing
- ☐ Stop-to-link ≤ 5 s p95 on the reference setup
- ☐ Cross-tenant access tests pass; no public bucket paths
- ☐ Extension installed from the Chrome Web Store starts a recording from any tab with click highlights visible in the output

V1 — weeks 15–30: compete

Order	Theme	Epics
1	Webcam bubble, quality presets, shortcuts, audio meters	1, 2
2	HLS ladder, sprites, captions pipeline, SSE status, full player	5, 6
3	Trim/cut editor with EDL rendering	7
4	Whisper transcription, transcript search	8, 11
5	Workspaces, roles, invites, shared library, folders, trash	11, 13
6	Password/expiry/invite links, oEmbed, Open Graph	9
7	Comments, reactions, views, basic analytics	10, 17
8	OAuth, TOTP, active sessions	12
9	Plans, local + international payments, usage metering	14
10	Notification centre and preferences; CDN; observability stack	15, 19

Exit criteria:

- ☐ 5 paying workspaces with ≥ 5 seats each
- ☐ Captions generated for 100% of recordings within $2\times$ duration
- ☐ Unique-viewer counts match server logs within 2%
- ☐ WCAG 2.2 AA audit passes on recorder, library and player

V2 — differentiators and scale

Theme	Epics	Trigger to start
AI summary, chapters, filler removal, translation	8	Transcription stable in V1
Public API, webhooks, Slack/Jira/GitHub/Notion	16	First team customer asks
Viewer insights, retention curves, CTA	10, 17	Teams using analytics weekly
SSO, SCIM, audit log, admin policies	12, 13	First enterprise deal
GPU workers, autoscaling, multi-region	5, 19	Queue age > 5 min at peak or transcription lagging
Swahili i18n, offline recording	20	Expansion beyond developer segment

13. Risks and open questions

The largest risks are browser capture inconsistencies and media cost; both are mitigated by early cross-browser testing and aggressive storage lifecycle rules.

Risks

Risk	Likelihood	Impact	Mitigation
Browser API differences break capture (Safari, Firefox)	High	High	Capability matrix tests in CI (Playwright on all engines); graceful degradation; per-browser mime selection
No system audio on macOS/Safari/Firefox disappoints users	High	Medium	Clear messaging before recording; tab-audio flow in the MVP extension; native helper later

Risk	Likelihood	Impact	Mitigation
Self-hosted MinIO: single-node disk failure or VPS outage loses or blocks media	Medium	High	RAID or erasure-coded drives; nightly off-site replication (<code>mc mirror</code>) to a second provider; disk alerts at 70%; restore drill monthly
MinIO VPS bandwidth saturated by playback	Medium	Medium	CDN caching of all segments and posters; long cache TTLs on immutable keys; monitor origin hit ratio
WebM chunk concatenation edge cases (pause/resume, codec switches)	Medium	High	Always re-mux through FFmpeg; golden-file tests with paused and long recordings
Whisper on CPU competes with FFmpeg on the worker VPS	High	Medium	Separate concurrency limit and lower priority for <code>Transcribe</code> ; <code>base</code> model; move to GPU host when queue age > 1 h
Mobile money operator API onboarding is slow or inconsistent (M-Pesa, Airtel Money, Mixx by Yas, HaloPesa)	High	Medium	Start merchant onboarding with each operator during MVP; launch with the first approved operator; <code>PaymentProvider</code> trait per operator; daily reconciliation job
Free tier (50 × 10 min at 1080p) drives storage cost	Medium	Medium	Inactive-video retention limits, lazy HLS ladder, per-workspace storage metrics, raw-chunk cleanup
Chrome Web Store review delays or rejects the extension	Medium	Medium	Minimal permissions (<code>activeTab</code> , <code>tabCapture</code> , <code>offline</code> , <code>storage</code>); privacy policy ready before submission; web app works without the extension

Risk	Likelihood	Impact	Mitigation
Abuse: hosting illegal or infringing content on public links	Medium	High	Abuse reports, takedown flow, public-link rate limits, V2 automated moderation
Single developer bus factor	High	High	This document, ADRs per major decision, infrastructure as code, runbooks, handoff checklist

Questions and decisions

- ✓ ~~Product name and domain. Decided: Sintade, domain still to register.~~
- ✓ ~~Object storage provider: self-hosted MinIO on the VPS, or Cloudflare R2 from day one? Decided: self-hosted MinIO on the VPS.~~
- ✓ ~~Who is the first target segment: individual developers, local businesses, or educators? Decided: developers and tech teams.~~
- ✓ ~~Free tier limits: are 25 recordings × 5 min at 720p right for the chosen segment? Decided: 50 recordings × 10 min at up to 1080p.~~
- ✓ ~~Pricing currency and model: USD per seat, TZS local plans, or both? Decided: TZS local plans.~~
- ✓ ~~Whisper hosting: CPU on the worker VPS, or a separate GPU host once volume justifies it? Decided: CPU on the worker VPS.~~
- ✓ ~~Is a browser extension needed earlier than V2 for the target segment (click highlights, system audio workarounds)? Decided: build it alongside the MVP.~~
- ✓ ~~Data residency: any customers requiring data stored in-country? Decided: no requirement.~~

Decision log

Decision	Choice	Alternatives considered
Architecture	Modular monolith + separate worker	Microservices (too much overhead for one developer)
Upload path	Presigned PUT per chunk, direct to storage	Proxy through API (bandwidth + memory cost), tus server

Decision	Choice	Alternatives considered
Job queue	Postgres SKIP LOCKED	Redis, NATS, RabbitMQ (extra infra)
Playback	MP4 fast-start first, HLS added async	HLS only (slower first play)
Local persistence	OPFS with IndexedDB fallback	Memory only (data loss on crash)
Chunk timeslice	2 s	1 s (more requests), 10 s (more loss window)
Object storage	Self-hosted MinIO on a VPS, CDN in front	Cloudflare R2, Backblaze B2
First segment	Developers and tech teams	Local businesses, educators, support/sales
Pricing	TZS local plans	USD per seat, both currencies
Payments	Direct mobile money operator APIs	Selcom, Pesapal, Flutterwave aggregators
Free tier	50 recordings, 10 min each, up to 1080p	25 × 5 min at 720p; 10 × 3 min; trial only
Transcription	whisper.cpp on CPU on the worker VPS	Separate GPU host, paid API, none
Browser extension	Built alongside the MVP	V1, V2
Data residency	No requirement	In-country only; enterprise option
Product name	Sintade (from cinta de casete)	Kanda and Reclo (both taken), Kioo, Ona, Onyesha

14. Progress tracker

The single source of truth for where work stands; whoever touches an item updates its Status and Notes (last change, open branch, blockers) before ending a session.

Rules: one row = one mergeable unit of work. **In progress** must name the branch in Notes. **Blocked** must name the blocker. **Done** means merged to **main**, deployed, and its acceptance criteria (section 15) pass.

Phase	Item	Epics	Status	Notes
MVP	Cargo workspace, core , platform crates	19	Not started	
MVP	Docker Compose dev env + CI pipeline	19	Not started	
MVP	Angular shell: ApiClient , auth interceptor, CapabilityService	20	Not started	
MVP	Identity: register, login, refresh rotation, logout	12	Not started	
MVP	Identity: email verification, password reset	12	Not started	
MVP	Personal workspace auto-created on signup	13	Not started	
MVP	Capture: source selection + device picker	1	Not started	
MVP	Capture: AudioMixer (mic + system audio)	1	Not started	
MVP	Capture: ChunkRecorder , controls, countdown	2	Not started	
MVP	Capture: OPFS ChunkStore + recovery dialog	3	Not started	
MVP	Ingest: recording, take, upload-session creation	4	Not started	
MVP	Ingest: presign, ack, status, finalize	4	Not started	
MVP	Client Uploader : streaming queue + retry	3, 4	Not started	
MVP	Worker binary, job queue, outbox relay	5, 19	Not started	
MVP	ProcessTake : concat, ffprobe, MP4 fast-start, poster	5	Not started	
MVP	Recording state machine + "ready" email	5, 15	Not started	
MVP	Share links: private / link / public	9	Not started	
MVP	Watch page, playback grants, basic player	6	Not started	
MVP	Library list, rename, trash	11	Not started	

Phase	Item	Epics	Status	Notes
MVP	Hardening: rate limits, headers, tenant-isolation tests	18	Not started	
MVP	Backups + production VPS deploy	19	Not started	
V1	Webcam bubble, quality presets, shortcuts, meters	1, 2	Not started	
V1	HLS ladder, sprites, SSE status, full player	5, 6	Not started	
V1	Trim/cut editor + EDL rendering	7	Not started	
V1	Whisper transcription + transcript search	8, 11	Not started	
V1	Workspaces, roles, invites, folders, trash restore	11, 13	Not started	
V1	Password/expiry/invite links, oEmbed, Open Graph	9	Not started	
V1	Comments, reactions, views, basic analytics	10, 17	Not started	
V1	OAuth, TOTP, active sessions	12	Not started	
V1	Plans, payments, usage metering	14	Not started	
V1	Notification centre, CDN, observability stack	15, 19	Not started	
MVP	Browser extension	2, 16	Not started	
V2	AI summary, chapters, filler removal, translation	8	Not started	
V2	Public API, webhooks, third-party integrations	16	Not started	
V2	Viewer insights, retention curves, CTA	10, 17	Not started	
V2	SSO, SCIM, audit log, admin policies	12, 13	Not started	
V2	GPU workers, autoscaling, multi-region	5, 19	Not started	
V2	Swahili i18n, offline recording	20	Not started	

15. User stories and acceptance criteria (MVP)

Each MVP story has testable criteria; a tracker row is **Done** only when every criterion under its story passes. V1/V2 stories are written in the same format before work on them starts.

Format: **ID — As a <role>, I want <capability>, so that <outcome>**. Criteria use Given/When/Then where behaviour is conditional.

Identity

US-01 — As a visitor, I want to sign up with email and password, so that I can record.

- ☐ Password ≥ 10 chars, checked against a breached-password list (top 100k)
- ☐ Duplicate email returns a generic message (no account enumeration)
- ☐ Personal workspace created in the same transaction as the user
- ☐ Verification email sent within 60 s; link valid 24 h, single use

US-02 — As a user, I want to log in and stay logged in, so that I don't re-enter credentials daily.

- ☐ Access cookie 15 min, refresh cookie 30 days, both `HttpOnly; Secure; SameSite=Lax`
- ☐ Given a refresh token is reused after rotation, when it is presented, then the whole session family is revoked
- ☐ 5 failed logins per IP+email per minute returns `429`

US-03 — As a user, I want to reset a forgotten password, so that I regain access.

- ☐ Reset link valid 1 h, single use; all sessions revoked on reset
- ☐ Response identical whether or not the email exists

Capture and recording

US-10 — As a creator, I want to choose screen, window or tab plus mic and system audio, so that I record exactly what I need.

- ☐ Mic list populated after permission; last choice remembered
- ☐ Given the browser lacks system audio, when the recorder opens, then the option is disabled with a one-line reason
- ☐ Denied permissions show recovery instructions per browser, not a blank screen

US-11 — As a creator, I want start, pause, resume and stop with a countdown, so that I control the take.

- ☐ 3-2-1 countdown, skippable with `Esc`
- ☐ Timer excludes paused time
- ☐ Given the user clicks the browser's "Stop sharing", then recording stops and upload completes as with the Stop button

US-12 — As a creator, I want my recording to survive a crash, so that I never lose work.

- ☐ Every chunk is in OPFS before its upload starts
- ☐ Given the tab is killed mid-recording, when the app is reopened, then a dialog offers Upload or Discard with date and duration
- ☐ Recovered recording plays with no gap longer than the last unflushed timeslice (≤ 2 s)

Upload and processing

US-20 — As a creator, I want uploading to happen while I record, so that my link is ready right after I stop.

- ☐ Stop $\rightarrow$ link ≤ 5 s p95 for 30 min at 1080p on 10 Mbps uplink
- ☐ Given the network drops for 60 s, then chunks queue locally and upload on reconnect with no loss
- ☐ Re-sent chunk with same hash is a no-op; different hash returns `409`

US-21 — As a creator, I want my recording to play in any browser, so that anyone can watch it.

- ☐ MP4 H.264/AAC with `faststart` produced for every take
- ☐ Output duration within 100 ms of recorded duration; seeking works end to end
- ☐ Poster generated; recording state becomes `ready`; creator gets a "ready" email
- ☐ Given processing fails 5 times, then state is `failed`, job is in DLQ, creator sees a retry button

Sharing and watching

US-30 — As a creator, I want to share a link with chosen visibility, so that the right people can watch.

- ☐ Link copied to clipboard on stop by default
- ☐ Private: only owner; link: anyone with URL; public: listed and indexable

- ☐ Changing visibility takes effect on the next playback request (≤ 15 min for already issued URLs)

US-31 — As a viewer, I want the video to start quickly, so that I don't give up.

- ☐ First frame ≤ 1.5 s p75 on 10 Mbps
- ☐ Speed 0.5–2×, fullscreen, keyboard seek ($\leftarrow/\rightarrow$ 5 s)
- ☐ Unauthorised viewer gets 404 (not 403) on private recordings

Library

US-40 — As a creator, I want to see, rename and delete my recordings, so that I stay organised.

- ☐ List newest first with thumbnail, title, duration, date; cursor pagination 24 per page
- ☐ Rename inline; delete moves to trash; links stop working immediately
- ☐ Trash purged after 30 days by `PurgeRecording` , including the storage prefix

16. Developer setup

A new developer must go from clone to a working local recording in under 30 minutes; if it takes longer, fixing the setup is the first task. The canonical copy of this section lives in the repo `README.md` ; this is the summary.

Prerequisites

Tool	Version	Purpose
Rust	stable, pinned in <code>rust-toolchain.toml</code>	Backend
<code>sqlx-cli</code>	matches <code>sqlx</code> in <code>Cargo.lock</code>	Migrations, offline query data
Node.js	LTS, pinned in <code>.nvmrc</code>	Angular
Docker + Compose	current	Postgres, MinIO, Mailpit locally
FFmpeg + ffprobe	≥ 6.0	Worker (also in worker image)

Tool	Version	Purpose
<code>just</code>	current	Task runner

Repository layout

```
sintade/
├─ crates/           # modules (section 5)
├─ bin/api, bin/worker
├─ migrations/
├─ web/              # Angular
├─ docs/
│   ├─ adr/          # architecture decision records
│   ├─ runbooks/     # deploy, rollback, incidents
│   └─ fixtures/     # golden media files for tests
├─ deploy/           # Dockerfiles, compose.prod.yml, CI helpers
├─ compose.yml       # local dependencies
├─ .env.example
└─ justfile
```

First run

```
git clone <repo-url> sintade && cd sintade
cp .env.example .env
just deps-up          # docker compose up -d: postgres, minio, mailpit
just db-migrate       # sqlx migrate run
just seed             # demo user demo@local.test / demo-password-123 +
sample recording
just api              # cargo run -p api      -> http://localhost:8080
just worker           # cargo run -p worker
just web              # cd web && npm ci && npm start ->
http://localhost:4200
```

Verify: log in as the demo user, record 10 s, stop, and the recording plays within a few seconds. Emails appear in Mailpit at `http://localhost:8025`; stored objects at MinIO console `http://localhost:9001`.

Environment variables

Variable	Example (local)	Notes
<code>DATABASE_URL</code>	<code>postgres://app:app@localhost:5432/sintade</code>	
<code>S3_ENDPOINT</code>	<code>http://localhost:9000</code>	MinIO locally and in production
<code>S3_BUCKET</code>	<code>sintade-dev</code>	Private bucket
<code>S3_ACCESS_KEY</code> , <code>S3_SECRET_KEY</code>	<code>minio</code> / <code>minio-secret</code>	Never committed
<code>PUBLIC_BASE_URL</code>	<code>http://localhost:4200</code>	Used in links and emails
<code>SESSION_SECRET</code>	64 random bytes, base64	Signs cookies
<code>DATA_ENC_KEY</code>	32 random bytes, base64	AES-256-GCM for TOTP secrets
<code>SMTP_URL</code>	<code>smtp://localhost:1025</code>	Mailpit locally
<code>FFMPEG_PATH</code>	<code>ffmpeg</code>	Worker only
<code>WORKER_CONCURRENCY</code>	<code>2</code>	Parallel jobs per worker
<code>RUST_LOG</code>	<code>info,api=debug,worker=debug</code>	

Everyday commands

Command	Does
<code>just check</code>	<code>cargo fmt --check</code> , <code>clippy -D warnings</code> , <code>sqlx prepare --check</code> , <code>npm run lint</code>
<code>just test</code>	Rust unit + integration (spins test DB), Angular unit
<code>just e2e</code>	Playwright on Chromium, Firefox, WebKit

Command	Does
<code>just migration <name></code>	New timestamped migration
<code>just reset</code>	Drop and recreate local DB + bucket

Troubleshooting

- `getDisplayMedia` fails on `http://` origins other than `localhost`: use `localhost` or HTTPS.
- `sqlx` compile errors after schema change: run `just db-migrate` then `cargo sqlx prepare --workspace`.
- Uploads fail with CORS errors: MinIO bucket CORS must allow `PUT` from `PUBLIC_BASE_URL` (`just deps-up` applies it).

17. Coding conventions

The codebase should read as if one person wrote it; CI enforces formatting and lints, and everything else here is enforced in review.

Rust

- `cargo fmt` default config; `clippy -D warnings` with `clippy::pedantic` allowed per-crate only where justified.
- One module crate = `domain/`, `app/`, `infra/`, `events.rs`, `lib.rs` exporting only the service trait, DTOs and events.
- `domain/` has no `async`, no `sqlx`, no I/O; invariants enforced in constructors (`Recording::new` returns `Result`).
- Newtype IDs from `core` (`RecordingId(Uuid)`); never pass raw `Uuid` across module boundaries.
- Errors: `thiserror` enums per module; convert to `AppError` at the handler boundary; `anyhow` only in binaries and tests.
- No `unwrap()` / `expect()` outside tests and startup config; `#!` `[deny(clippy::unwrap_used)]` in library crates.
- SQL: `sqlx::query!` / `query_as!` only (compile-checked); every tenant query includes `workspace_id`.

- Transactions: service methods own the transaction; repositories take `&mut PgConnection`.
- Logging: `#[tracing::instrument(skip_all, fields(recording_id = %id))]` on service methods; never log emails, tokens or passwords.
- Time: `time::OffsetDateTime` in UTC; inject `Clock` for testability.

```
// crates/catalog/src/app/service.rs – reference shape for every module
service
#[async_trait]
pub trait CatalogService: Send + Sync {
    async fn rename(&self, ctx: &RequestCtx, id: RecordingId, title: Title) -
> Result<RecordingDto, CatalogError>;
}

pub struct CatalogServiceImpl { pool: PgPool, outbox: Outbox, clock: Arc<dyn
Clock> }

#[async_trait]
impl CatalogService for CatalogServiceImpl {
    #[tracing::instrument(skip_all, fields(recording_id = %id))]
    async fn rename(&self, ctx: &RequestCtx, id: RecordingId, title: Title) -
> Result<RecordingDto, CatalogError> {
        let mut tx = self.pool.begin().await?;
        let mut rec = repo::get_for_update(&mut tx, ctx.workspace_id,
id).await?.ok_or(CatalogError::NotFound)?;
        ctx.require(Permission::EditRecording, &rec)?;
        rec.rename(title, self.clock.now());
        repo::save(&mut tx, &rec).await?;
        self.outbox.push(&mut tx, RecordingRenamed::from(&rec)).await?;
        tx.commit().await?;
        Ok(rec.into())
    }
}
```

Angular / TypeScript

- Standalone components, signals for local state, `OnPush` everywhere; RxJS for streams (capture, uploads, SSE).
- `strict: true`; no `any` (ESLint error); API DTOs generated from the backend OpenAPI spec (`utoipa`) into `web/src/app/api/`.

- `capture/` is framework-free TS (no Angular imports) so the MVP browser extension reuses it unchanged.
- Feature folders: `feature/<name>/{components, services, routes.ts}`; shared UI in `shared/ui`.
- All user-facing strings through `$localize` from day one (Swahili later).

Naming

Thing	Convention	Example
Tables	plural snake_case	<code>share_links</code>
Columns	snake_case; timestamps end <code>_at</code> ; booleans read as questions	<code>revoked_at</code> , <code>allow_download</code>
Events	PastTense PascalCase	<code>TakeFinalized</code>
Job kinds	PascalCase verb-noun	<code>ProcessTake</code>
REST paths	plural nouns, kebab-case	<code>/recordings/{id}/share-links</code>
Rust crates	single lowercase word	<code>ingest</code>
Angular files	kebab-case with type suffix	<code>chunk-store.service.ts</code>

Git workflow

- Trunk-based: short-lived branches off `main`, named `<type>/<tracker-item>` (e.g. `feat/ingest-finalize`).
- Conventional Commits (`feat:`, `fix:`, `refactor:`, `docs:`, `test:`, `chore:`).
- Every PR: linked tracker row, updated acceptance checklist, green CI, migration reviewed for lock safety.
- Squash merge; `main` is always deployable; feature flags (`workspace_policies.flags.jsonb`) hide unfinished UI.
- Never force-push `main`; tags `vYYYY.MM.DD-N` mark production deploys.

Migrations

- Forward-only, one concern per file, prefixed by module:
`20260921_1200_ingest_add_chunks.sql`.
- Zero-downtime rules: add nullable column → backfill in a job → add constraint `NOT VALID` → `VALIDATE`; `CREATE INDEX CONCURRENTLY` in its own migration.
- Never edit a migration that has reached production.

18. Test strategy

Tests guard the two things a new developer is most likely to break without noticing: tenant isolation and the capture-to-playback media path.

Level	Scope	Tools	Runs
Unit (Rust)	<code>domain/</code> invariants, state machine, EDL maths, slug entropy	<code>cargo test</code> , <code>proptest</code>	Every commit
Unit (TS)	<code>capture/</code> logic with mocked <code>MediaRecorder</code> , <code>uploader</code> <code>retry/backoff</code> , <code>ChunkStore</code>	Vitest / Jest	Every commit
Integration (Rust)	Services + real Postgres + MinIO; handlers via <code>tower::ServiceExt::oneshot</code>	<code>sqlx::test</code> , <code>testcontainers</code>	Every PR
Tenant isolation	Every endpoint called as a user of workspace B on workspace A's resources → must return <code>404</code>	Generated test table over the route list	Every PR
Media pipeline	Golden files through <code>ProcessTake</code> , <code>BuildHls</code> , <code>RenderEdit</code> ; assert duration, codecs, seekability via <code>ffprobe</code>	<code>docs/fixtures/*.webm</code> , <code>ffprobe</code> JSON	Every PR touching <code>media</code>
Cross-browser E2E	Record (fake media), upload, play on Chromium, Firefox, WebKit	Playwright with <code>--use-fake-device-for-media-stream</code> , <code>--use-fake-ui-for-media-stream</code>	Every PR to <code>main</code>

Level	Scope	Tools	Runs
Resilience	Kill tab mid-recording, drop network 60 s, worker crash mid-job	Playwright + network offline emulation; chaos script	Nightly
Load	200 concurrent uploads, 1,000 concurrent viewers	k6	Before each phase release
Security	Dependency audit, secret scan, OWASP ZAP baseline	cargo audit, cargo deny, npm audit, gitleaks, ZAP	Nightly
Manual	Real system audio on Windows Chrome, Safari recording, 4K 60 fps	Release checklist	Before each production deploy

Golden media fixtures (docs/fixtures/)

File	Tests
vp9_opus_30s.webm	Baseline Chrome output
vp8_opus_paused.webm	Pause/resume timestamp gaps
h264_aac_safari_20s.mp4	Remux-only path
vp9_no_audio.webm	Silent recordings
vp9_opus_60min.webm (generated, not committed)	Long-duration memory and time budget
truncated_last_chunk.webm	Crash recovery with partial final chunk
not_a_video.webm	ffprobe rejection path

Rules

- A bug fix lands with a test that fails before the fix.
- Coverage is not a target; the tenant-isolation table and golden files are mandatory.
- Flaky tests are fixed or quarantined within 24 h, with a tech-debt entry (section 21).

19. Deployment runbook

Every production change goes through GitHub Actions from a tagged `main` commit; nobody deploys from a laptop. Detailed step-by-step runbooks live in `docs/runbooks/`.

Environments

Env	Host	Data	Deploys
Local	Developer machine, Docker Compose	Seed data	Manual
Staging	Small VPS	Anonymised copy, refreshed weekly	Every merge to <code>main</code>
Production	API VPS + worker VPS + managed/self-run Postgres	Real	Manual approval of a tag

Pipeline

```

flowchart LR
    PR[Pull request] --> CI[CI: check, test,<br/>e2e, audit]
    CI --> M[Merge to main]
    M --> IMG[Build images<br/>api, worker, web]
    IMG --> STG[Deploy staging<br/>+ migrate]
    STG --> SMK[Smoke tests]
    SMK --> TAG[Tag release]
    TAG --> APR[Manual approval]
    APR --> PRD[Deploy prod<br/>migrate, rolling restart]
  
```

Images are tagged with the git SHA and pushed to GHCR; production pulls by SHA, never `latest`.

Secrets

- Stored in GitHub Actions environment secrets (staging, production) and written to the VPS as `/etc/sintade/env` (mode `0600`) at deploy.
- Rotation: `SESSION_SECRET` supports two keys (current + previous) for zero-downtime rotation; storage keys rotated every 90 days.

- An inventory of every secret, where it is used, and who can access it lives in the handoff checklist (section 22), never the values.

Deploy steps (production)

1. Confirm staging smoke tests green for the tagged SHA.
2. Approve the `deploy-production` job.
3. Job runs `sqlx migrate run` against production (migrations must be backward-compatible with the running version).
4. Worker: stop taking new jobs (`SIGTERM`), finish or re-queue current job, restart on new image.
5. API: rolling restart, one instance at a time, health check `/healthz` and `/readyz` before the next.
6. Web: upload static bundle to CDN; purge `index.html` only.
7. Watch dashboards 15 min: 5xx rate, p95 latency, queue age, job failures.

Rollback

- App: re-run the deploy job with the previous tag (images are immutable).
- Database: migrations are forward-only; roll back by deploying the previous app version (compatible by rule) and fixing forward. Restore from PITR only for data corruption, following `docs/runbooks/restore.md`.
- Web: re-upload the previous bundle.

Incident response

Symptom	First check	Runbook
Recordings stuck in <code>processing</code>	Queue age, worker logs, scratch disk	<code>worker-stuck.md</code>
Uploads failing	Storage credentials/CORS, presign errors, rate limits	<code>uploads-failing.md</code>
Playback 403/404	Clock skew on signing host, CDN token config	<code>playback-errors.md</code>
API 5xx spike	Recent deploy, DB connections, slow queries	<code>api-errors.md</code>

Symptom	First check	Runbook
Disk full	Worker scratch, Postgres WAL, logs	<code>disk-full.md</code>

After any incident affecting users: a short post-mortem in `docs/incidents/YYYY-MM-DD-<slug>.md` (what happened, impact, cause, fix, follow-ups added to section 21).

20. Architecture decision records

Every decision that is expensive to reverse gets an ADR in `docs/adr/`, so a new developer can see why, not only what; the decision log in section 13 is the index, ADRs hold the reasoning.

When to write one

- Choosing or replacing a technology, provider or protocol.
- Changing a module boundary, dependency rule or data ownership.
- Anything a future developer might reasonably want to undo.

Template (`docs/adr/NNNN-title.md`)

```
# NNNN. <Decision title>

- Status: Proposed | Accepted | Superseded by NNNN | Deprecated
- Date: YYYY-MM-DD
- Deciders: <names>

## Context
What problem, which constraints (cost, time, skills, browser limits).

## Options considered
1. <Option A> — pros / cons
2. <Option B> — pros / cons

## Decision
What we chose and the one-sentence reason.

## Consequences
```

What becomes easier, what becomes harder, what we must now do.

Revisit when

The measurable trigger that should reopen this decision.

Initial ADRs to write

No.	Title	Revisit when
0001	Modular monolith with separate media worker	A module needs independent scaling beyond the worker
0002	Presigned direct-to-storage chunk upload	Storage provider lacks presigned PUT or CORS control
0003	Postgres SKIP LOCKED job queue + transactional outbox	Queue throughput > 5k jobs/min or lock contention visible
0004	OPFS local chunk persistence with IndexedDB fallback	Browser support regresses or quota issues exceed 1% of sessions
0005	2-second MediaRecorder timeslice	Upload request count becomes a cost or rate-limit problem
0006	Fast-start MP4 first, HLS asynchronously	Median recording length exceeds 20 min
0007	Cookie sessions with rotating refresh tokens	Public API or mobile clients need bearer tokens
0008	Self-hosted whisper.cpp for transcription	Transcription queue age > 1 h or accuracy complaints
0009	Self-hosted MinIO object storage	Storage VPS disk > 70% full or any durability incident
0010	Direct mobile money operator integrations for TZS plans	Operator onboarding blocks launch or reconciliation takes > 1 day/month

21. Known issues and tech-debt log

Anything a new developer would otherwise discover the hard way goes here: bugs not yet fixed, shortcuts taken deliberately, and platform limitations. Newest entries at the top; remove a row only when it is fixed and the fix is merged.

ID	Type	Severity	Description	Workaround / plan	Added
KI-006	Tech debt	Low	Billing is a stub returning free-tier <code>Entitlements</code> in MVP	Replace in V1 billing work; interface already final	Design
KI-005	Tech debt	Low	Single worker host; no autoscaling	Add hosts manually by queue age; autoscaling in V2	Design
KI-004	Limitation	Medium	<code>MediaRecorder</code> WebM output lacks duration/cues metadata	Always re-mux through FFmpeg before serving	Design
KI-003	Limitation	Low	No screen recording on mobile browsers	View-only on mobile; message on recorder page	Design
KI-002	Limitation	Medium	Firefox and Safari cannot capture system audio	Pre-recording notice; suggest Chrome/Edge	Design
KI-001	Limitation	Medium	macOS/Linux Chrome captures tab audio only, not full-system audio	Guide users to share a tab when audio matters; extension (MVP) for the tab-audio flow; native helper later	Design

Entry rules: one row per issue; link the PR, branch or incident file in the description; severity High means users can lose data or access, Medium means a feature is degraded, Low means cosmetic or internal.

22. Handoff checklist

A handoff is complete when the incoming developer can deploy to production alone and every account is owned by the project, not a person. The outgoing developer completes their half before leaving; the incoming developer completes theirs in the first week.

Access inventory (names and owners only, never secret values)

Asset	Where	Owner account	Transferred
Source repository	GitHub organisation		[]
CI/CD secrets	GitHub Actions environments		[]
Container registry	GHCR		[]
Domain + DNS	Registrar / Cloudflare		[]
VPS hosts (API, worker, staging)	Hosting provider		[]
Postgres (prod, staging) + backups	Host / managed provider		[]
Object storage buckets + keys	MinIO (storage VPS)		[]
CDN	Cloudflare / Bunny		[]
Email provider	SES / Postmark		[]
Payment providers + webhook secrets	Mobile money operators (M-Pesa, Airtel Money, Mixx by Yas, HaloPesa)		[]
Error tracking + monitoring	Sentry, Grafana		[]
Password manager vault	Shared project vault		[]

All accounts must be registered to a project/company email, with at least two admins.

Outgoing developer

- ☐ Progress tracker (section 14) current: every row's Status and Notes accurate, open branches named
- ☐ Unmerged branches either merged, deleted, or described in Notes with remaining work
- ☐ Known issues log (section 21) updated with everything in your head
- ☐ ADRs written for any undocumented decision

- ☐ `README.md` first-run verified on a clean machine in the last 30 days
- ☐ Runbooks cover every manual operation you have performed in production
- ☐ Access inventory filled in; credentials moved into the shared vault
- ☐ Recorded walkthrough (using the platform itself): architecture tour, capture engine, worker pipeline, a full deploy — 60 minutes total
- ☐ Available for questions for an agreed period after leaving

Incoming developer — first week

- ☐ Day 1: read sections 1–6 and 14; get all access; complete local first run (section 16)
- ☐ Day 2: record, share and watch a recording locally; read sections 7–10; trace one recording through the code from `ChunkRecorder` to `ProcessTake`
- ☐ Day 3: run the full test suite and E2E; read sections 17–18; fix one small item from section 21
- ☐ Day 4: deploy to staging yourself; read section 19 and every runbook
- ☐ Day 5: shadowed or solo production deploy of a small change; rotate one secret; confirm you can restore a backup to staging
- ☐ End of week: list every confusing thing you hit and fix the docs for each

Handoff record

Date	From	To	Notes